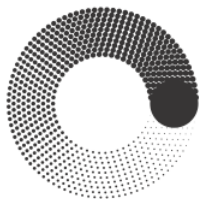


МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ



**Факультет Информационных технологий
кафедра Информатики и информационных технологий**

направление подготовки 09.002 «Информационные системы и технологии»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

БАКАЛАВРСКАЯ РАБОТА

Тема: Разработка сайта для настольно-ролевой игры

Исполнитель Гайдарь М.Д.

(Фамилия И.О.)

(Подпись)

Руководитель Кандрашина М.А.

(Фамилия И.О., степень, звание)

(Подпись)

«ДОПУЩЕНО К ЗАЩИТЕ»

Зав. кафедрой ИиИТ:

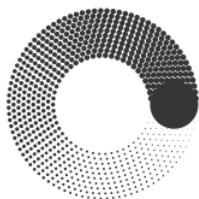
(Подпись)

к.т.н. Е.В. Булатников

Москва

2026

МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ



**Факультет Информационных технологий
кафедра Информатики и информационных технологий**

направление подготовки 09.002 «Информационные системы и технологии»

УТВЕРЖДАЮ

Зав. каф., к.т.н. Е.В. Булатников

« ____ » _____ 2026 г. _____

(Подпись)

ЗАДАНИЕ НА БАКАЛАВРСКУЮ РАБОТУ

Студент Гайдарь Максим Денисович группа 221-3711

(Фамилия, Имя, Отчество)

Тема Разработка сайта для настольно-ролевой игры

Утверждена приказом по Университету № _____ от «__» января _____ г.

1. Срок представления работы к защите «__» июня _____ г.

2. Исходные данные для выполнения работы: Техническое задание, техническая литература по теме ВКР

3. Содержание бакалаврской работы: Введение, аналитическая часть, конструкторская часть, практическая часть, заключение

4. Перечень графического материала Презентация

5. Дата выдачи задания: «__» января _____ г.

6. Руководитель: _____ /М.А. Кандрашина/

(Подпись) (И.О. Фамилия)

7. Задание к исполнению принял: М. Д. Гайдарь/

(Подпись) (И.О. Фамилия)

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ



Факультет Информационных технологий
кафедра Информатики и информационных технологий

направление подготовки 09.002 «Информационные системы и технологии»

КАЛЕНДАРНЫЙ ПЛАН

Студент Гайдарь Максим Деинсович **группа** 221-3711
(Фамилия, Имя, Отчество)

Тема ВКР: Разработка сайта для настольно-ролевой игры

№ п/п	Наименование этапа ВКР	Срок выполнения этапа	Отметка о выполнении
	Исследование предметной области	26.07.2025-29.08.2025	Выполнено
	Разработка технического задания	30.07.2025-06.08.2025	Выполнено
	Эскизное проектирование	07.08.2025-13.09.2025	Выполнено
	Техническое проектирование	14.08.2025-27.10.2025	Выполнено
	Рабочее проектирование	28.09.2025-24.11.2025	Выполнено
	Разработка	25.09.2025-06.05.2026	Выполнено
	Тестирование	07.02.2026-10.05.2026	Выполнено
	Приемосдаточные испытания	21.04.2026-31.05.2026	Выполнено

Руководитель М.А. Кандрашина

(Фамилия И.О., степень, звание)

(Подпись)

Зав. каф. ИиИТ /Е.В. Булатников/

(Подпись)

« » 2026 г.

Аннотация

В данной выпускной квалификационной работе рассматривается процесс создания веб-сайта для настольной ролевой системы E'Magios Core. Проект представляет собой многофункциональную веб-платформу, включающую базу данных игровых материалов, редактор персонажей, систему бросков кубов, интеграцию с Discord.

Структура работы представлена введением, тремя главами, заключением, списком литературы и приложением.

Во введении обоснована актуальность темы проекта.

Первая глава посвящена изучению предметной области, системному анализу, поставленным цели и задачам.

Во второй главе описывается проектирование структуры сайта, базы данных, пользовательских интерфейсов, а также механизмов авторизации через Google, хранения данных в Firebase и парсинга контента из Obsidian.

Третья глава посвящена процессу разработки и программной реализации базы данных, редактора персонажей, системы бросков кубов, интеграции с Discord.

Заключение содержит итоги проделанной работы и возможности дальнейшего развития проекта.

Abstract

This final qualifying work examines the process of creating a website for the tabletop role-playing system E'Magios Core. The project is a multifunctional web platform that includes a database of game materials, a character editor, a dice rolling system, Discord integration, and a lobby for online user interaction.

The structure of the work consists of an introduction, three chapters, a conclusion, a list of references, and an appendix.

The introduction explains the relevance of the project topic.

The first chapter is devoted to the study of the subject area, system analysis, and the formulation of the goals and objectives.

The second chapter describes the design of the website structure, the database, the user interfaces, as well as the mechanisms of Google authentication, data storage in Firebase, and content parsing from Obsidian.

The third chapter is devoted to the development process and software implementation of the database, the character editor, the dice rolling system, Discord integration, and the lobby for playing on the website.

The conclusion contains the results of the completed work and the possibilities for further development of the project.

Реферат

Название: Разработка веб-платформы для настольной ролевой системы.

Объем пояснительной записки составляет 68 страниц, включает 11 рисунков, 2 таблицы, 6 листингов, 3 приложения. При написании дипломной работы использовалось 16 источников.

Ключевые слова: веб-сайт, веб-платформа, настольная ролевая система, Firebase, Google Auth, база данных, редактор персонажей, броски кубов, Discord, Obsidian, JSON, JavaScript.

Оглавление

Оглавление

БАКАЛАВРСКАЯ РАБОТА	1
ЗАДАНИЕ НА БАКАЛАВРСКУЮ РАБОТУ	2
КАЛЕНДАРНЫЙ ПЛАН	3
Аннотация	4
Abstract	5
Реферат	6
Оглавление	7
Введение	8
Глава 1. Аналитическая часть	10
1.1 Цели и задачи	10
1.2 Анализ схожих проектов	12
1.3 Обзор выбранных технологий для разработки	15
Глава 2. Конструкторская часть	20
2.1 Разработка дизайн-документа	20
2.2 Описание архитектуры и механизмов работы сайта	25
Глава 3. Практическая часть	30
Заключение	42
Список литературы	43

Введение

Актуальность разрабатываемого проекта обусловлена ростом интереса к настольным ролевым системам и одновременным увеличением потребности в цифровых инструментах, сопровождающих игровой процесс. Современным пользователям уже недостаточно только текстового описания правил: востребованы удобная навигация по материалам, быстрый доступ к справочной информации, хранение данных персонажей, инструменты для бросков кубов и средства удаленного взаимодействия участников. Особенно это важно для авторских ролевых систем, где контент постоянно развивается, дополняется и требует структурированного представления в единой цифровой среде.

Проект E'Magios Core представляет собой веб-платформу для настольной ролевой системы, объединяющую несколько функциональных направлений. Сайт включает в себя базу данных игровых сущностей, гиперссылочную навигацию по материалам, редактор персонажей, систему бросков кубов, авторизацию через Google, хранение пользовательских данных в Firebase, интеграцию с Discord через вебхуки, а также бета-версию лобби для онлайн-взаимодействия игроков. Дополнительной особенностью проекта является использование Obsidian как источника исходного контента с последующим парсингом и преобразованием материалов для публикации на сайте.

Целью данной выпускной квалификационной работы является разработка многофункционального веб-сайта для поддержки настольной ролевой системы E'Magios Core, обеспечивающего удобный доступ к игровому контенту и предоставляющего пользователям интерактивные инструменты для подготовки и сопровождения игрового процесса. Для достижения поставленной цели решаются задачи проектирования структуры сайта, организации базы данных игровых материалов, реализации пользовательского интерфейса, настройки механизмов авторизации и хранения данных, а также разработки вспомогательных игровых модулей.

Проект реализуется с использованием HTML, CSS и JavaScript без применения тяжелых фреймворков, что позволяет сохранить простоту

архитектуры, высокую скорость загрузки и удобство сопровождения. Для работы с пользовательскими данными применяются сервисы Firebase, включая аутентификацию и облачное хранение. Такой подход обеспечивает практическую значимость разработки и создает основу для дальнейшего расширения функциональности сайта, включая развитие онлайн-взаимодействия, улучшение игровых инструментов и масштабирование цифровой экосистемы проекта E'Magios Core.

Глава 1. Аналитическая часть

1.1 Цели и задачи

Целью данного проекта является создание многофункционального веб-сайта для настольной ролевой системы E'Magios Core, обеспечивающего удобный доступ к игровым материалам, хранение пользовательских данных и использование интерактивных инструментов для сопровождения игрового процесса. Сайт ориентирован на объединение справочного контента, редактора персонажей, системы бросков кубов и средств онлайн-взаимодействия в рамках единой цифровой платформы. Для достижения этой цели необходимо решить следующие задачи:

1. Провести исследование предметной области цифровых веб-инструментов для настольных ролевых систем, изучив основные подходы к организации игрового контента, пользовательских данных и вспомогательных сервисов.
2. Выполнить анализ аналогичных веб-ресурсов и сервисов для настольных ролевых игр с целью выявления их сильных и слабых сторон.
3. Спроектировать архитектуру веб-приложения, включающую модули навигации по материалам, базы данных, авторизации, редактора персонажей, системы бросков кубов и онлайн-взаимодействия пользователей.
4. Разработать веб-сайт, реализующий:
 - а. Структурированную базу данных игровых сущностей, включающую заклинания, школы магии, эффекты, навыки, действия и другие материалы системы Гибкую систему навыков игроков на основе ScriptableObject с визуальными и звуковыми эффектами.
 - б. Удобную гиперссылочную навигацию по разделам сайта и связанным игровым объектам Визуализацию игрового поля и действий с использованием пулов объектов для оптимизации.

- c. Редактор персонажей с возможностью сохранения данных в аккаунте пользователя, а также экспортом и импортом через JSON
 - d. Систему авторизации через Google и хранение пользовательских данных в Firebase.
 - e. Модуль бросков кубов с возможностью использования характеристик персонажа при расчете результатов
 - f. Интеграцию с Discord через вебхуки для отправки результатов бросков.
 - g. Механизм парсинга и преобразования контента из Obsidian для публикации на сайте.
 - h. Бета-версию лобби для онлайн-взаимодействия пользователей в рамках игрового процесса.
5. Провести тестирование программного продукта для проверки корректности работы основных модулей, стабильности системы и соответствия поставленным требованиям.

1.2 Анализ схожих проектов

В данном разделе рассматриваются веб-проекты, реализующие функциональные элементы, схожие с концепцией разрабатываемого сайта для настольной ролевой системы E'Magios Core. Анализ позволяет определить наиболее удачные подходы к организации контента, навигации, работе с пользовательскими данными и вспомогательными игровыми инструментами. В качестве аналогов были выбраны ресурсы dnd.su, ttg.club и Long Story Short, так как они представляют разные подходы к цифровой поддержке настольных ролевых игр.

Сайт dnd.su является русскоязычным онлайн-справочником по Dungeons & Dragons 5-й редакции. Его сильной стороной выступает большой объем структурированного игрового контента: классы, заклинания, черты, снаряжение, бестиарий и дополнительные материалы. Для пользователя особенно важны развитая система фильтрации, поиск по игровым сущностям и развитая гиперссылочная связность между разделами. Такой подход показывает значимость хорошо организованной базы данных и удобной навигации для быстрого доступа к правилам. В то же время основной акцент dnd.su сделан именно на справочном наполнении, тогда как интерактивные пользовательские инструменты не являются центральной частью платформы. Для проекта E'Magios Core данный ресурс представляет интерес прежде всего как пример построения крупной базы знаний по настольной системе.

Проект ttg.club также является онлайн-справочником по Dungeons & Dragons и демонстрирует более современный подход к организации интерфейса. На сайте представлены разделы с классами, видами, чертами, заклинаниями, снаряжением, магическими предметами, бестиариумом и глоссарием правил. Дополнительно реализованы отдельные инструменты, например калькулятор характеристик и токенизатор. Особый интерес для анализа представляет удобная система поиска и фильтрации контента, позволяющая быстро отбирать игровые объекты по заданным параметрам. Именно такой подход к фильтрам и

представлению справочной информации близок к базе данных E'Magios Core. Однако ttg.club ориентирован на поддержку уже существующей популярной системы, тогда как разрабатываемый сайт решает иную задачу: он служит цифровой платформой для собственной авторской ролевой системы и совмещает справочные материалы с пользовательскими модулями.

Сервис Long Story Short показывает другой важный вектор развития цифровых инструментов для настольных ролевых игр. Основное назначение данного проекта связано с интерактивным листом персонажа, встроенными формулами бросков, поддержкой цифрового использования листа и интеграцией бросков с внешними сервисами, включая Discord и Telegram. Такой подход демонстрирует ценность персонализированных пользовательских инструментов, которые сопровождают игровой процесс, а не только предоставляют справочную информацию. Для проекта E'Magios Core данный аналог особенно важен как пример того, как редактор персонажа и система бросков могут быть связаны в единой цифровой среде. Вместе с тем Long Story Short в большей степени сосредоточен на листе персонажа и индивидуальном использовании, тогда как разрабатываемый сайт дополнительно включает собственную базу данных, систему парсинга контента из Obsidian и бета-версию игрового лобби.

По сравнению с рассмотренными аналогами сайт E'Magios Core сочетает в себе сразу несколько направлений. Он объединяет справочную базу данных игровых материалов, гиперссылочную навигацию между сущностями, редактор персонажей с сохранением данных через Google-аккаунт и Firebase, систему бросков кубов с возможностью подтягивания характеристик персонажа, интеграцию с Discord через вебхуки, а также механизм подготовки и публикации контента из Obsidian. Кроме того, в проекте реализуется бета-версия лобби для онлайн-взаимодействия пользователей, что выводит сайт за пределы обычного справочника или цифрового листа персонажа.

Проведенный анализ позволяет сделать вывод, что существующие проекты обычно развиваются по одному из двух направлений: либо как крупные справочные базы правил, либо как набор персональных игровых инструментов.

Разрабатываемый сайт E'Magios Core отличается тем, что объединяет оба подхода в рамках одной платформы. Это позволяет использовать сильные стороны аналогов, сохраняя при этом уникальность проекта как цифровой среды для авторской настольной ролевой системы.

По результатам анализа можно выделить следующие ориентиры для разработки:

1. Необходимость структурированной базы данных с удобной фильтрацией и быстрым поиском;
2. Важность гиперссылочной навигации между разделами и игровыми сущностями;
3. Целесообразность интеграции редактора персонажей и системы бросков кубов;
4. Полезность облачного хранения пользовательских данных и авторизации;
5. Перспективность объединения справочного и интерактивного функционала в рамках одного веб-ресурса.

1.3 Обзор выбранных технологий для разработки

В В данном разделе описываются особенности технологий, которые были выбраны для реализации веб-платформы E'Magios Core, их преимущества и применение в контексте разработки. Для создания проекта были использованы технологии веб-разработки, облачные сервисы хранения данных, средства автоматизации обработки контента и инструменты публикации статического сайта.

1.3.1 HTML, CSS и JavaScript

В качестве основной технологической базы для разработки сайта были выбраны HTML5, CSS3 и JavaScript. Данные технологии являются стандартом современной веб-разработки и позволяют реализовать функциональный и доступный веб-интерфейс без использования тяжелых фреймворков и сборщиков. Рабочая структура проекта включает отдельные HTML-страницы, таблицы стилей и JavaScript-модули, отвечающие за навигацию, базу данных, редактор персонажей, авторизацию, броски кубов и лобби.

Использование HTML5 позволило сформировать логичную структуру страниц и представить материалы ролевой системы в виде набора связанных разделов. CSS3 применяется для создания единой темной визуальной темы, адаптивной боковой навигации, таблиц, модальных окон, карточек и элементов управления. JavaScript используется как основное средство реализации прикладной логики сайта, включая генерацию сайдбара, фильтрацию базы данных, работу с гиперссылками, обработку пользовательских действий и динамическое обновление интерфейса.

Одним из ключевых преимуществ выбранного подхода является простота сопровождения проекта. Отказ от сложных внешних зависимостей позволяет сохранить прозрачную архитектуру, высокую скорость загрузки страниц и удобство ручной доработки. Такой подход особенно важен для проекта,

ориентированного на долговременное развитие и регулярное пополнение контента.

1.3.2 Firebase

Для реализации авторизации и облачного хранения пользовательских данных в проекте используется платформа Firebase. В рамках сайта она применяется для входа через Google, хранения профилей пользователей, персонажей, истории бросков кубов и данных игрового лобби.

Firebase был выбран благодаря тому, что предоставляет готовую инфраструктуру для веб-приложений и позволяет быстро внедрить аутентификацию и облачную базу данных без необходимости развертывания собственного серверного решения. Использование Firebase Authentication обеспечивает удобный вход через Google-аккаунт, а Firestore позволяет хранить структурированные пользовательские данные и синхронизировать их между сессиями и устройствами.

Дополнительным преимуществом Firebase является поддержка работы в реальном времени. Это особенно важно для бета-версии игрового лобби, где используются общие комнаты, сообщения, приглашения и синхронизация действий пользователей. Таким образом, Firebase обеспечивает не только сохранение данных, но и основу для развития онлайн-функциональности проекта. Рабочая область данного инструмента продемонстрирована на рисунке 1.1.

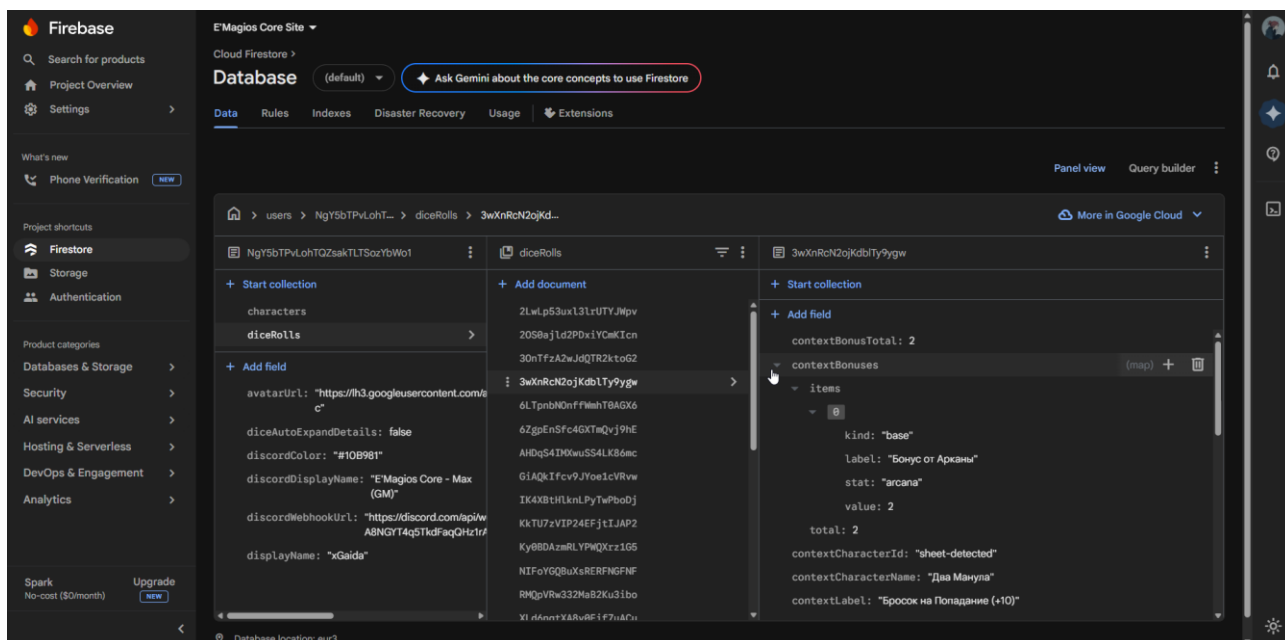


Рисунок 1.1 — Рабочая область Firebase

1.3.3 Python и автоматизация обработки контента

Для подготовки и обновления контента сайта были выбраны Python-скрипты, обеспечивающие парсинг материалов из Obsidian Vault и преобразование их в формат, пригодный для публикации на сайте. Такой подход был выбран в связи с тем, что исходные материалы ролевой системы хранятся в формате Markdown и регулярно дополняются.

Python применяется для автоматического парсинга игровых сущностей, таких как заклинания, школы магии, эффекты, действия, навыки и другие элементы системы. Также с его помощью реализовано преобразование Markdown-файлов в HTML-страницы с сохранением структуры заголовков, списков, таблиц и гиперссылок. Это позволяет поддерживать единый источник правды для контента и уменьшает количество ручной работы при обновлении сайта.

Преимуществом Python в данном случае является простота написания служебных сценариев, удобство работы со строками и файлами, а также возможность быстро адаптировать парсеры под изменение структуры исходных

материалов. Использование автоматизации значительно повышает согласованность контента и снижает вероятность ошибок при переносе данных из Obsidian в веб-формат.

1.3.4 Obsidian

Важную роль в проекте играет приложение Obsidian, которое используется как основная среда хранения и редактирования исходных материалов ролевой системы. В Obsidian размещаются Markdown-файлы, содержащие главы книг правил, описания заклинаний, школ магии, эффектов и других игровых объектов.

Выбор Obsidian обусловлен удобством работы с Markdown, поддержкой внутренней связности материалов и возможностью централизованного редактирования контента. Благодаря этому все игровые данные сначала создаются и обновляются в удобной авторской среде, а затем автоматически преобразуются для публикации на сайте. Такой подход делает процесс сопровождения системы более организованным и позволяет отделить редактирование контента от веб-разработки. Рабочая область данного инструмента представлена на рисунке 1.2.

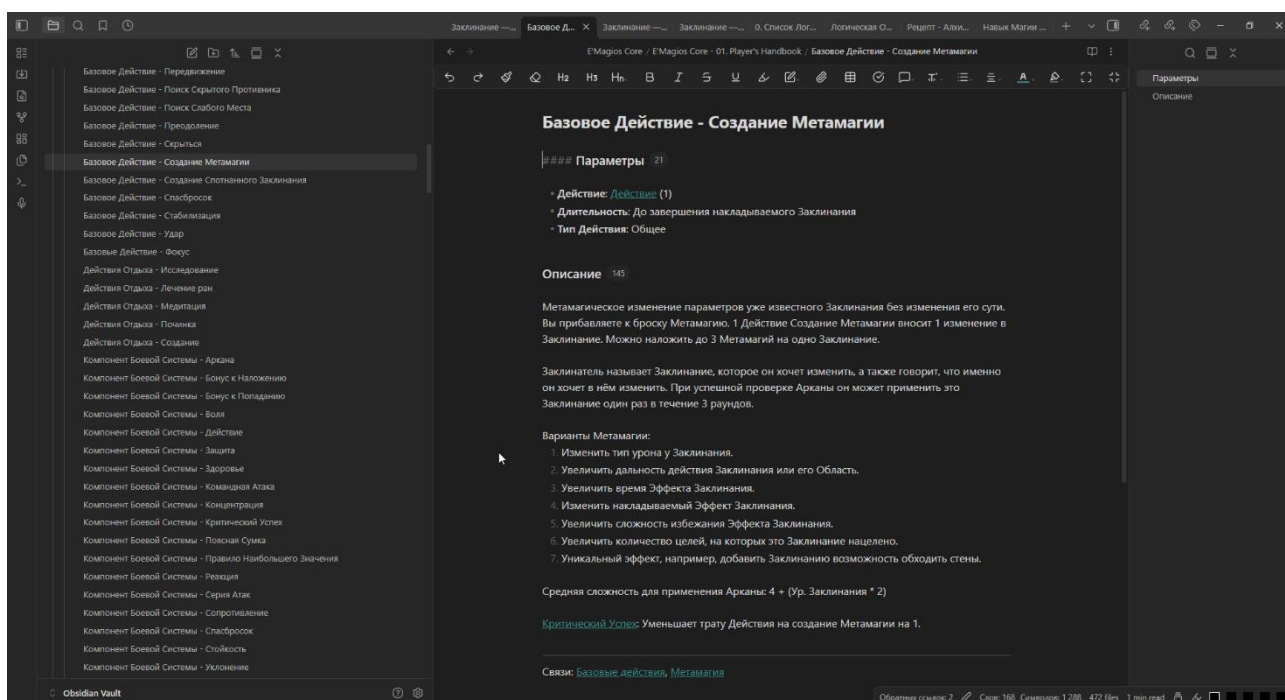


Рисунок 1.2 — Рабочая область Obsidian

1.3.5 Git и GitHub Pages

Для контроля версий и публикации проекта используются Git и GitHub Pages. Git обеспечивает хранение истории изменений, удобную организацию разработки и возможность безопасного внесения доработок в кодовую базу. GitHub Pages используется как платформа для размещения статического веб-сайта.

Выбор GitHub Pages обусловлен тем, что проект построен как статический сайт без обязательного серверного рендеринга. Это позволяет упростить процесс публикации, уменьшить требования к инфраструктуре и обеспечить быстрый доступ к сайту через браузер. Использование Git и GitHub Pages также делает проект удобным для сопровождения, тестирования и поэтапного развития.

Глава 2. Конструкторская часть

2.1 Разработка дизайн-документа

2.1.1. Общая информация о проекте

Проект E'Magios Core представляет собой многостраничную веб-платформу, предназначенную для цифровой поддержки авторской настольной ролевой системы. Основная задача сайта заключается в объединении справочного контента, пользовательских игровых инструментов и онлайн-функций в рамках единой среды. В отличие от обычного статического сайта с текстовыми страницами, проект сочетает в себе навигацию по книгам правил, структурированную базу данных игровых сущностей, редактор персонажей, модуль бросков кубов, интеграцию с облачным хранением и бета-режим сетевого взаимодействия пользователей.

Целевая аудитория проекта включает игроков и мастеров настольных ролевых игр, которым необходим быстрый доступ к правилам, заклинаниям, эффектам, действиям, навыкам и другим элементам системы. Кроме того, сайт ориентирован на пользователей, которые хотят хранить своих персонажей в аккаунте, выполнять броски кубов с учетом характеристик персонажа, использовать Discord-интеграцию и взаимодействовать с другими участниками в рамках игрового лобби.

Сайт разрабатывается как модульная клиентская система на основе HTML, CSS и JavaScript. В качестве общей точки инициализации используется модуль `common.js`, который подключает основные инфраструктурные механизмы проекта. Навигационная структура задается через объект `BOOKS` в модуле `books.js`, а построение бокового меню реализуется функцией `initSidebar()` в модуле `sidebar.js`. За разграничение доступа к защищенным разделам отвечают функции `checkAccess()`, `showPasswordModal()` и `initPasswordProtection()` в модуле `access.js`.

С точки зрения пользовательской логики проект состоит из нескольких ключевых подсистем. Модуль `db.js` реализует базу данных игровых материалов с вкладками, фильтрами, сортировкой и детальными карточками объектов. Модуль `character-editor.js` отвечает за создание, сохранение и редактирование персонажей. Модуль `profile.js` реализует работу с профилем пользователя и настройками Discord. Модуль `lobby.js` обеспечивает работу бета-версии сетевого лобби, включая создание комнат, чат, совместные броски и общий холст. Для авторизации и подключения к Firebase используется модуль `auth.js`, а параметры подключения вынесены в `firebase-config.js`. Более детально ознакомиться со взаимодействием модулей можно на рисунке 2.1.

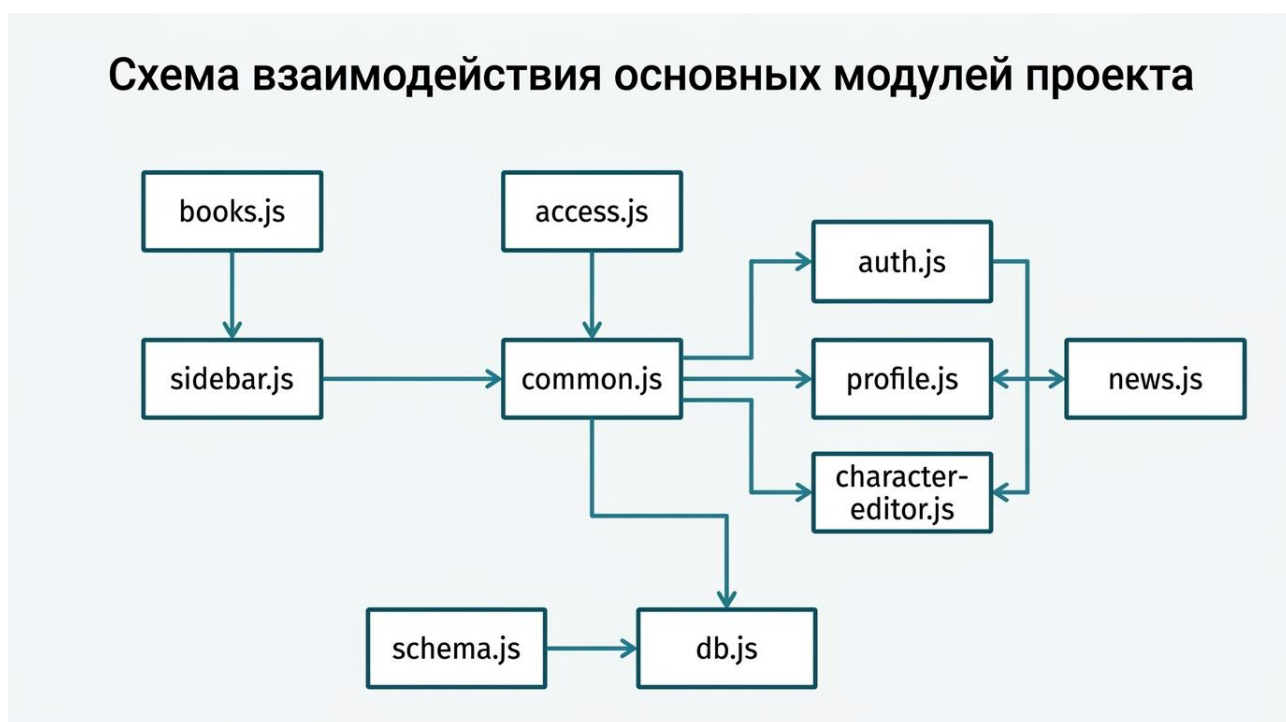


Рисунок 2.1 — Схема взаимодействия основных модулей

Дополнительной важной особенностью проекта является наличие отдельного конвейера подготовки контента. Исходные материалы хранятся в Obsidian Vault в формате Markdown, после чего преобразуются в HTML-страницы и JSON-файлы при помощи Python-скриптов. В частности, за конвертацию глав отвечает функция `convert_markdown_to_html()` из

convert_md_to_html.py, а за генерацию структурированных данных для базы знаний отвечают скрипты parse_spells.py, parse_schools.py, parse_effects.py, parse_actions.py, parse_skills.py и другие. Таким образом, дизайн-документ проекта фиксирует не только структуру пользовательского интерфейса, но и полную схему жизненного цикла контента от авторского хранилища до публикации на сайте.

2.1.2 Определение основных особенностей проекта

Основные особенности проекта E'Magios Core определяются не только тематикой ролевой системы, но и выбранными архитектурными решениями, обеспечивающими удобство использования, масштабируемость и функциональную целостность платформы. Все особенности перечислены на картинке 2.2.

Основные функциональные возможности платформы

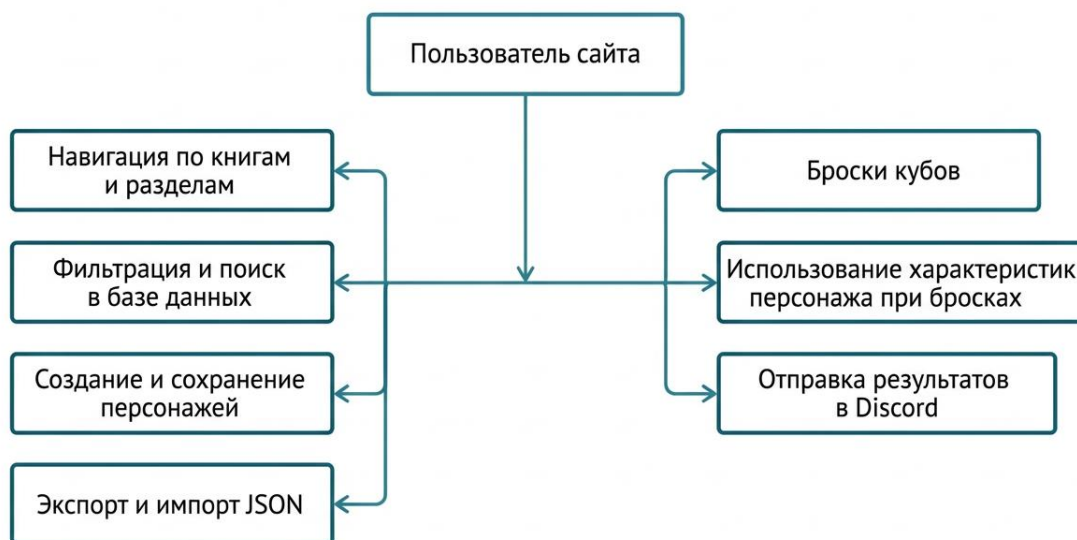


Рисунок 2.2 — Основные функциональные возможности платформы

Одной из ключевых особенностей сайта является data-driven навигация по книгам и разделам. Общая структура книг и глав задается в модуле `books.js` через объект `BOOKS`, а затем автоматически используется функцией `initSidebar()` из `sidebar.js` для построения бокового меню. Это позволяет централизованно управлять навигацией по материалам системы и исключить дублирование разметки на страницах.

Второй важной особенностью является наличие защищенного контента. Доступ к отдельным книгам реализуется через функции `checkAccess()`, `checkPassword()`, `initPasswordProtection()` и `isBookLocked()` из модуля `access.js`. Такой подход позволяет разделить публичные и ограниченные материалы без изменения общей структуры сайта.

Третьей особенностью выступает модульная база данных игровых сущностей. В проекте используется декларативная схема `DB_SCHEMAS` из модуля `schema.js`, которая задает колонки и фильтры для различных сущностей. На ее основе модуль `db.js` реализует загрузку данных, фильтрацию, сортировку и открытие карточек объектов. Важную роль в этом модуле играют функции `loadAllData()`, `ensureDbDataLoaded()`, `switchTab()`, `initializeDynamicFilters()`, `filterAndDisplaySpells()`, `filterAndDisplaySchools()` и `openDbDetailModal()`. Благодаря этому база данных сайта не сводится к простому набору таблиц, а представляет собой полноценную прикладную подсистему.

Следующей особенностью является редактор персонажей, связанный с облачным хранением данных. В модуле `character-editor.js` реализованы функции `collectFormData()`, `fillForm()`, `saveCharacter()`, `performAutosave()`, `renderCharactersList()` и `subscribeCharacters()`. Редактор позволяет создавать нескольких персонажей, сохранять их в учетной записи пользователя, а также выполнять экспорт и импорт в формате JSON. Дополнительно редактор связан с базой данных заклинаний и школ магии, что обеспечивает более тесную интеграцию игровых инструментов.

Отдельной особенностью проекта является встроенный модуль бросков кубов. Он реализован в `common.js` через функции `parseRollExpression()`,

`rollDiceExpression()`, `handleDiceRollCommand()` и `addDiceResultToHistory()`. В отличие от простого генератора случайных значений, данный модуль поддерживает обработку формул, сохранение истории бросков и использование характеристик выбранного персонажа через механизм `applyCharacterBonusIfNeeded()`.

С проектом тесно связана и интеграция с Discord. В модуле `profile.js` пользователь может сохранить параметры интеграции при помощи `saveUserProfile()` и проверить их через `sendDiscordTestMessage()`. После этого результаты бросков могут автоматически отправляться в Discord с использованием функции `sendDiceResultToDiscord()` из `common.js`. Это расширяет рамки веб-платформы и делает ее частью внешней цифровой экосистемы.

Еще одной значимой особенностью является наличие бета-версии игрового лобби. Модуль `lobby.js` реализует создание и подключение к комнатам через функции `createLobby()` и `joinByInviteCode()`, управление участниками через `ensureMembership()`, текстовый чат через `bindChatForm()` и `sendRollMessage()`, а также совместный холст через `initCanvas()` и `pushStroke()`. Таким образом, проект выходит за рамки справочного сайта и приближается к формату онлайн-платформы для сопровождения игрового процесса.

Наконец, важной особенностью является использование Obsidian как источника исходного контента и Python-скриптов как слоя преобразования данных. Функции `convert_wikilinks_in_text()` и `resolve_wikilink()` из `link_resolver.py` обеспечивают преобразование внутренних ссылок Obsidian в ссылки сайта, а функция `create_html_page()` из `convert_md_to_html.py` участвует в формировании итоговых HTML-страниц. Благодаря этому поддерживается единый источник правды для содержательной части проекта.

2.2 Описание архитектуры и механизмов работы сайта

2.2.1. Общая архитектура системы

Архитектура проекта E'Magios Core построена по принципу многостраничного веб-сайта с клиентским слоем расширения функциональности. Базовый уровень системы представлен набором HTML-страниц, каждая из которых отвечает за определенный раздел сайта. Поверх статической структуры работает JavaScript-слой, обеспечивающий динамическую генерацию интерфейса, загрузку данных, обработку пользовательских действий, авторизацию и онлайн-взаимодействие. Отдельно от клиентской части существует Python-конвейер подготовки контента, преобразующий материалы из Obsidian Vault в конечные HTML- и JSON-артефакты. Более наглядно с переводом контента из Obsidian в Firebase можно ознакомиться на рисунке 2.3.

Общая архитектура веб-платформы E'Magios Core

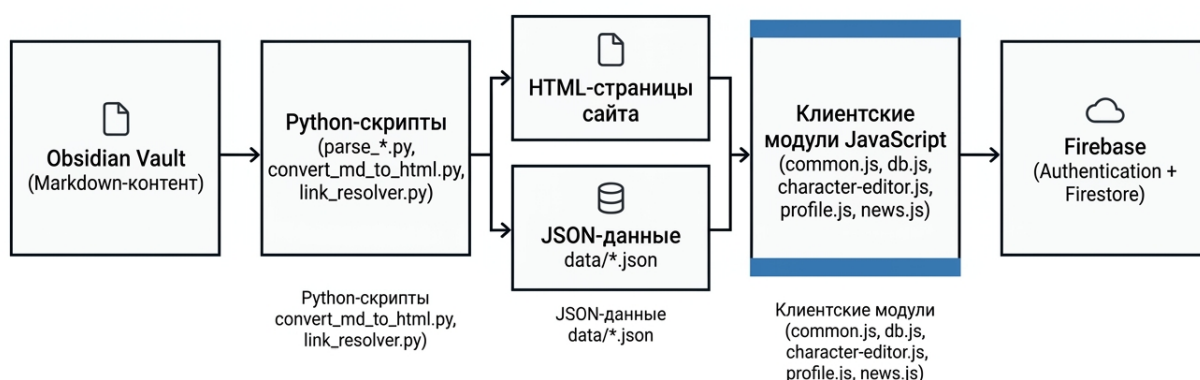


Рисунок 2.3 — Общая архитектура веб-платформы

Центральным инфраструктурным модулем выступает `common.js`. Он выполняет роль общего bootstrap-слоя и связывает между собой основные подсистемы. Через него подключаются функции защиты контента из `access.js`, функции навигации из `sidebar.js`, а также механизмы прокрутки и вспомогательные компоненты интерфейса. Кроме того, в `common.js` реализуются универсальные механизмы загрузки страниц, работы с бросками кубов и открытия карточек базы данных из разных частей сайта.

Важным уровнем архитектуры является контентная модель. Структура книг и глав задается в `books.js`, а схема сущностей базы данных задается в `schema.js` через `DB_SCHEMAS`. Таким образом, навигация и представление данных описываются декларативно, а конкретные интерфейсные модули используют эти описания при построении страниц.

Функциональные модули проекта разделены по назначению. Модуль `db.js` отвечает за страницу базы данных, модуль `character-editor.js` за редактор персонажей, `profile.js` за профиль и пользовательские настройки, `news.js` за отображение новостей, а `lobby.js` за сетевую beta-часть проекта. Такой подход обеспечивает изоляцию ответственности между частями системы и упрощает сопровождение проекта.

Отдельный слой архитектуры связан с облачными сервисами. В модуле `auth.js` реализованы функции `initFirebaseApp()`, `initCharacterEditorAuth()` и `initDiceAuth()`, обеспечивающие инициализацию Firebase и передачу состояния авторизации в клиентские модули. Для хранения данных используются возможности Firebase Authentication и Firestore. Благодаря этому проект поддерживает облачное хранение персонажей, профилей, истории бросков и данных лобби без собственного серверного backend-уровня.

Не менее важной частью архитектуры является конвейер подготовки контента. Скрипт `convert_md_to_html.py` отвечает за конвертацию Markdown-глав в HTML, а `link_resolver.py` за преобразование Obsidian-ссылок во внутренние ссылки сайта. Параллельно с этим скрипты семейства `parse_*.py` формируют JSON-файлы, которые затем загружаются в `db.js`. Таким образом,

архитектура проекта состоит из двух взаимосвязанных уровней: уровня публикации контента и уровня его клиентского использования.

В результате общая архитектура системы может быть охарактеризована как гибридный подход, сочетающий статическую публикацию материалов, модульную клиентскую логику и облачные сервисы хранения данных. Такое решение хорошо соответствует тематике проекта, поскольку позволяет одновременно поддерживать большой объем структурированного контента и предоставлять пользователям интерактивные инструменты.

2.2.2. Проектирование пользовательских механизмов.

Пользовательские механизмы проекта проектировались таким образом, чтобы все ключевые функции сайта были доступны в едином интерфейсном пространстве и логически дополняли друг друга.

Первым базовым механизмом выступает навигация и управление доступом. Навигация строится функцией `initSidebar()` на основе данных из BOOKS, что позволяет отображать структуру книг и активные разделы автоматически. Для взаимодействия с разделами используется функция `toggleBook()`, управляющая раскрытием списка глав. За разграничение доступа отвечает модуль `access.js`, в котором функции `checkAccess()` и `initPasswordProtection()` определяют, должен ли пользователь видеть содержимое защищенного раздела, а `showPasswordModal()` формирует интерфейс ввода пароля. Подробнее о навигации можно увидеть на рисунке 2.4.

Схема навигации и доступа к разделам

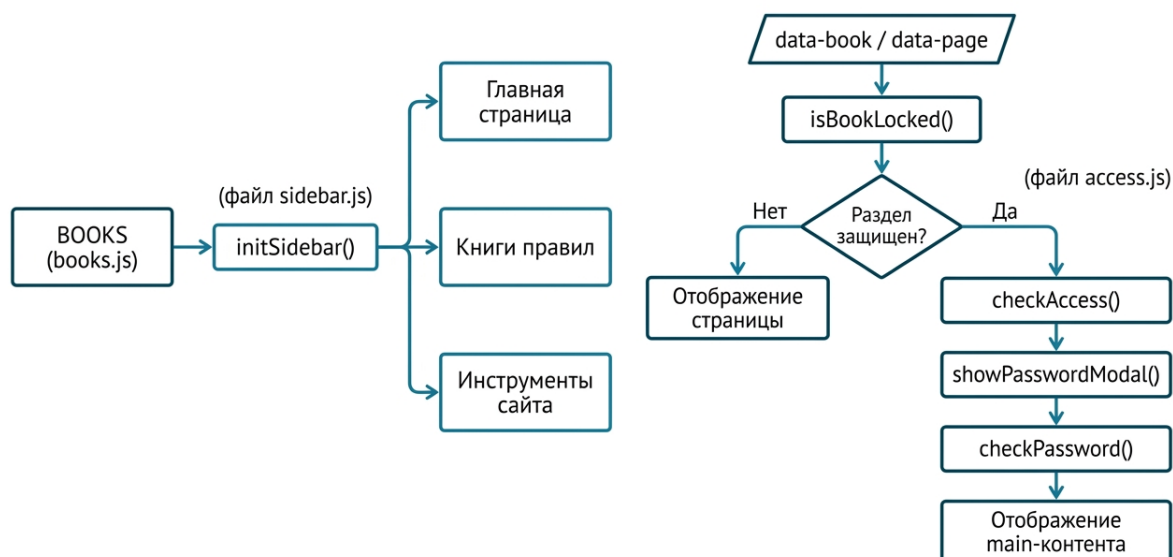


Рисунок 2.4 — Схема навигации и доступа к разделам

Вторым важным механизмом является база данных игровых сущностей. В ней центральную роль играет модуль `db.js`, который загружает JSON-данные и приводит их к пользовательскому виду. Функции `loadAllData()` и `ensureDbDataLoaded()` отвечают за получение данных, `switchTab()` за переключение между вкладками, `initializeDynamicFilters()` за формирование фильтров, а `filterAndDisplaySpells()`, `filterAndDisplaySchools()`, `filterAndDisplayEffects()` и аналогичные функции за отображение отфильтрованных наборов объектов. Открытие детальных представлений реализуется через `openDbDetailModal()` и набор функций вида `showSpellPage()`, `showSchoolPage()` и `showEffectPage()`. Такой подход обеспечивает единообразие интерфейса для различных сущностей при сохранении различий в их структуре.

Третьим механизмом выступает редактор персонажей. В модуле `character-editor.js` функция `collectFormData()` собирает структуру данных персонажа из формы, `fillForm()` заполняет интерфейс сохраненными данными, а `saveCharacter()` и `performAutosave()` реализуют ручное и автоматическое сохранение в Firestore. Функция `renderCharactersList()` отвечает за вывод списка сохраненных

персонажей, а `subscribeCharacters()` за синхронизацию редактора с облачным хранилищем. Отдельно предусмотрены механизмы импорта и экспорта JSON, что позволяет использовать редактор не только как онлайн-инструмент, но и как средство локальной работы с персонажами.

Следующим пользовательским механизмом является система бросков кубов. В проекте она выполняет не только роль вспомогательного инструмента, но и связывает между собой профиль пользователя, редактор персонажей и внешние интеграции. Функция `parseRollExpression()` разбирает строковое выражение броска, функция `rollDiceExpression()` вычисляет результат, а `handleDiceRollCommand()` объединяет обработку команды, применение контекста и формирование итоговой записи. Если для броска используется информация о персонаже, включается функция `applyCharacterBonusIfNeeded()`. Затем результат сохраняется функцией `addDiceResultToHistory()`, что обеспечивает историю бросков и повторный доступ к ранее выполненным действиям.

Отдельного внимания заслуживает механизм пользовательского профиля и Discord-интеграции. Модуль `profile.js` использует `loadUserProfile()` для загрузки данных пользователя, `saveUserProfile()` для их сохранения и `sendDiscordTestMessage()` для проверки корректности webhook-настроек. Эти параметры впоследствии используются модулем бросков, где функция `sendDiceResultToDiscord()` может автоматически отправлять результат в Discord. В результате профиль пользователя выполняет роль центральной точки настройки облачных и внешних интеграций.

Все перечисленные механизмы проектировались как взаимосвязанные модули. Пользователь может перейти из навигации к справочному контенту, затем к базе данных, открыть карточку сущности, использовать редактор персонажей, выполнить бросок с учетом характеристик, сохранить результат в истории и при необходимости отправить его в Discord или использовать в лобби.

Именно эта связность является одной из ключевых проектных особенностей платформы.

Глава 3. Практическая часть

3.1. Организация проекта и подготовка контента

3.1.1. Структура веб-проекта и общая инициализация

Практическая реализация E'Magios Core выполнена в формате многостраничного веб-приложения, состоящего из статических HTML-страниц, общего набора стилей, JavaScript-модулей и структурированных JSON-данных. Такой подход соответствует назначению проекта: сайт должен одновременно выполнять роль электронной библиотеки правил и набора интерактивных инструментов для игроков настольной ролевой системы. Основной контент системы представлен в виде книг правил, поэтому для него важны простая публикация, быстрый доступ и независимость от серверной генерации страниц.

Репозиторий разделен на несколько смысловых зон. В корне находятся основные страницы приложения: профиль пользователя, новости, база данных, редактор персонажей, список лобби и комната лобби. Разделы книг вынесены в отдельные каталоги `phb`, `spellbook`, `master`, `craftbook` и `rumors`. В них размещаются HTML-файлы отдельных глав. Каталог `data` хранит справочные данные в формате JSON: заклинания, школы магии, эффекты, действия, навыки, архетипы, базовые статьи, компоненты боя, компоненты ремесла, профессии, специализации и рецепты. Отдельные Python-скрипты используются для генерации этих файлов из исходных материалов.

Клиентская логика разделена по назначению. Файл `common.js` содержит общие функции сайта, включая загрузчик страниц, панель бросков кубов и вспомогательные обработчики. Модуль `sidebar.js` отвечает за боковую навигацию, `access.js` - за клиентское ограничение доступа к закрытым книгам,

`db.js` - за интерактивную базу знаний, `schema.js` - за декларативное описание колонок и фильтров, `character-editor.js` - за редактор персонажей, `auth.js` - за авторизацию, `profile.js` - за профиль пользователя, а `lobby.js` - за онлайн-лобби.

В проекте не используется отдельный backend-сервер. Страницы, стили, скрипты и JSON-файлы могут быть опубликованы на статическом хостинге, например GitHub Pages. Пользовательские данные, которым требуется хранение между устройствами или синхронизация в реальном времени, вынесены в Firebase Authentication и Cloud Firestore. Такая модель позволяет сохранить простую инфраструктуру сайта и при этом реализовать интерактивные функции, необходимые для цифрового сопровождения настольной игры.

Общая инициализация страниц реализована через модуль `common.js`, который подключается на страницах сайта и импортирует основные вспомогательные модули. Эта точка входа обеспечивает единое поведение разных типов страниц: книг, базы данных, редактора персонажей и лобби. Через нее подключаются проверка доступа, генерация боковой панели, плавная прокрутка и общие элементы интерфейса.

при загрузке защищенной книги `common.js` выполняет раннюю проверку доступа. Если страница относится к закрытой книге и пользователь еще не вводил пароль, основной элемент `main` скрывается до завершения проверки. Это предотвращает кратковременное отображение закрытого содержимого до появления модального окна. Данная логика особенно важна для статического сайта, где все HTML-страницы физически доступны в репозитории, а ограничение доступа работает на уровне пользовательского интерфейса.

Также в `common.js` реализованы общие элементы, которые используются на разных страницах: индикатор загрузки, обработчики фильтров, функции скачивания JSON, панель бросков кубов и переходы к сущностям базы данных. Такой подход уменьшает дублирование, так как одинаковые действия не переписываются отдельно для редактора, базы и страниц книг.

3.1.2. Конвертация Markdown-страниц в HTML

Исходные тексты E'Magios Core ведутся в Obsidian Vault в формате Markdown. Этот формат удобен на этапе разработки настольной системы, поскольку поддерживает быстрое редактирование текста, внутренние ссылки, заголовки, списки и таблицы. Для публикации на сайте материалы преобразуются в HTML-страницы, совместимые с общей структурой веб-проекта.

Конвертация выполняется скриптом `'convert_md_to_html.py'`. Он читает Markdown-файл, построчно анализирует содержимое и преобразует основные конструкции в HTML-разметку. Заголовки становятся тегами `'h2'`, `'h3'` и `'h4'`, списки преобразуются в `'ul'` и `'li'`, таблицы - в `'table'`, а выделение жирным и курсивом заменяется на соответствующие HTML-теги. Для заголовков дополнительно формируется идентификатор, который можно использовать как якорь для навигации.

После преобразования содержимое помещается в HTML-шаблон страницы. В шаблоне задаются кодировка, viewport, заголовок страницы, подключение `'styles.css'`, атрибуты `'data-book'` и `'data-chapter'` в теге `'body'`, контейнер для боковой панели и подключение `'common.js'`. Таким образом каждая сгенерированная глава сразу становится частью общей навигационной и визуальной системы сайта.

Отдельное внимание уделено ссылкам Obsidian. В исходных материалах активно используются wikilinks, которые удобны внутри Obsidian, но не работают напрямую в браузере. Поэтому при конвертации ссылки преобразуются в URL сайта или в ссылки на базу данных. Это позволяет сохранять связность материалов между книгами, правилами и справочными сущностями.

3.1.3. Формирование JSON-справочников

Для интерактивных разделов сайта одного HTML-представления недостаточно. Базе знаний, редактору персонажей и фильтрам требуются структурированные данные, которые можно сортировать, фильтровать и связывать между собой. Поэтому часть исходных Markdown-материалов преобразуется в JSON-файлы каталога ``data``.

Для разных типов сущностей используются отдельные парсеры: ``parse_spells.py``, ``parse_schools.py``, ``parse_effects.py``, ``parse_actions.py``, ``parse_skills.py``, ``parse_archetypes.py``, ``parse_recipes.py`` и другие. Каждый парсер ориентирован на структуру конкретного типа заметок. Например, парсер заклинаний извлекает название, действие, тип действия, дистанцию, цель, длительность, тип урона, школу, источник, описание и дополнительные параметры. Парсеры школ магии и рецептов извлекают другие наборы полей, соответствующие их роли в системе.

Модуль ``link_resolver.py`` используется как общий слой преобразования ссылок. Он удаляет служебную разметку `wikilinks` там, где требуется чистый текст, и формирует HTML-ссылки там, где связь должна быть кликабельной. Например, ссылка на эффект может быть преобразована в переход к соответствующей карточке базы данных, а ссылка на раздел правил - в переход к базовой статье. Благодаря этому данные в JSON остаются пригодными не только для отображения, но и для навигации между связанными объектами.

Такой конвейер обеспечивает единый источник правды для проекта. Автор продолжает работать с материалами в Obsidian, а сайт получает оптимизированные HTML-страницы и JSON-справочники. Это снижает риск рассинхронизации между текстом правил и интерактивной базой, поскольку обновление данных выполняется через парсеры, а не ручным редактированием JSON-файлов.

3.2. Навигация, доступ и база знаний

3.2.1. Структура боковой навигации и ограничение доступа

Навигация сайта реализована через боковую панель, которая отображает главные разделы, книги и инструменты. Ручное дублирование одинакового меню в каждой HTML-странице было бы неудобным, так как при добавлении новой главы потребовалось бы изменять множество файлов. Поэтому структура книг вынесена в модуль `'books.js'`, а HTML-разметка меню формируется динамически в `'sidebar.js'`.

В `'books.js'` каждая книга описывается объектом с названием, признаком закрытого доступа и списком глав. Каждая глава содержит идентификатор, отображаемое название и путь к HTML-файлу. Эти данные используются не только для построения бокового меню, но и для определения заголовка закрытой книги при запросе пароля. Такой подход делает структуру книг централизованной: изменение состава глав выполняется в одном месте.

На страницах книг в теге `'body'` используются атрибуты `'data-book'` и `'data-chapter'`. Модуль `'sidebar.js'` считывает эти значения, определяет текущую книгу и главу, раскрывает активный раздел и подсвечивает соответствующую ссылку. Если страница находится во вложенном каталоге, модуль корректирует относительные пути к главной странице, новостям, лобби, редактору и базе данных. Благодаря этому одна функция генерации меню работает как на корневых страницах, так и на страницах глав.

Боковая навигация выполняет не только функцию перехода между страницами, но и функцию ориентации пользователя в большом объеме правил. Поскольку сайт содержит несколько книг и десятки глав, автоматическая подсветка текущего положения снижает нагрузку на пользователя и делает работу с материалами ближе к структуре цифрового справочника.

Часть материалов сайта предназначена для ограниченного просмотра. В первую очередь это относится к мастерским разделам и дополнительным книгам,

которые не должны открываться случайно всем пользователям. Для этого реализовано клиентское ограничение доступа через модуль `'access.js'`. Оно не является полноценной серверной защитой, но выполняет задачу интерфейсного контроля для статического сайта.

Основой проверки является значение в `'localStorage'`. Если пользователь успешно ввел пароль, в браузере сохраняется флаг доступа. При повторном открытии защищенных страниц модуль проверяет этот флаг и не требует повторного ввода пароля. Если флаг отсутствует, основной контент страницы скрывается, а пользователю показывается модальное окно ввода пароля.

После успешного ввода пароля функция `'checkPassword'` записывает флаг доступа, удаляет модальное окно и возвращает отображение основного содержимого. Также предусмотрена кнопка сброса доступа, которая удаляет значение из локального хранилища и перезагружает страницу. Это полезно при тестировании и при демонстрации сайта разным пользователям.

Ограничение доступа реализовано осознанно как UX-механизм. Поскольку сайт размещается как статический набор файлов, HTML-контент технически доступен в репозитории и не может считаться защищенным на уровне сервера. В тексте дипломной работы это важно фиксировать явно: механизм скрывает материалы в интерфейсе, но не заменяет серверную авторизацию и выдачу контента по ролям.

3.2.2. Загрузка данных интерактивной базы

Интерактивная база знаний реализована в файле `'db.js'`. Ее задача состоит в том, чтобы предоставить пользователю удобный доступ к игровым сущностям: заклинаниям, школам магии, эффектам, действиям, навыкам, архетипам, базовым правилам, рецептам и компонентам ремесла. В отличие от обычных HTML-страниц, база должна поддерживать фильтрацию, сортировку, вкладки и переходы между связанными объектами.

При первой необходимости база загружает JSON-файлы из каталога ``data``. Для этого используется функция ``loadAllData``, которая параллельно вызывает загрузчики отдельных наборов данных. Параллельная загрузка сокращает время ожидания пользователя, поскольку браузер может запрашивать несколько файлов одновременно. Чтобы не выполнять повторные запросы при каждом переходе или открытии карточки, используется общий промис ``dbDataPromise``.

Каждый загрузчик получает данные через ``fetch``. Для запросов указан параметр ``cache: 'no-store'``, что особенно важно во время разработки: после повторного запуска парсеров браузер должен получить свежий JSON-файл, а не устаревшую кэшированную версию. При ошибке загрузки соответствующий массив заменяется пустым массивом, что позволяет интерфейсу продолжать работу и не ломать всю страницу из-за одного неудачного файла.

Поскольку сайт не использует серверную базу данных, вся логика выборки выполняется в браузере. Для текущего объема справочников это решение является достаточным и упрощает публикацию проекта. Пользователь получает быстрые фильтры и сортировку, а разработчик сохраняет возможность обновлять данные простым изменением JSON-файлов.

3.2.2. Загрузка данных интерактивной базы

Чтобы не дублировать описание интерфейса для каждой сущности, структура колонок и фильтров вынесена в файл ``schema.js``. В нем задаются поля, которые должны отображаться в таблицах и всплывающих карточках, а также фильтры, доступные пользователю. Такой подход делает базу знаний расширяемой: при добавлении нового типа сущности достаточно описать его схему и подключить соответствующий набор данных.

Фильтрация работает через временное состояние. Пользователь открывает модальное окно фильтров, выбирает нужные значения, а затем применяет изменения. До нажатия кнопки применения основная таблица не

перестраивается. Это делает поведение интерфейса предсказуемым и позволяет пользователю отменить изменения, если он выбрал неверные параметры.

Для просмотра полной информации о сущности используются модальные карточки. Карточка выводит параметры объекта, описание и связанные сущности. Связи между объектами являются важной частью базы знаний: из заклинания можно перейти к школе магии, из школы - к связанным школам или учебным заклинаниям, из описания эффекта - к соответствующему правилу. Для сохранения открытой карточки при переходах используется ``sessionStorage`` и параметры URL.

В результате база знаний становится не просто таблицей JSON-данных, а навигационным слоем над материалами E'Magios Core. Пользователь может изучать правила через книги или через связанные карточки, переходя от конкретного заклинания к механике, школе, эффекту и другим зависимым сущностям.

3.3. Пользовательские инструменты

3.3.1. Авторизация и профиль пользователя

Инструменты, связанные с персональными данными, требуют авторизации пользователя. Для этого на сайте используется Firebase Authentication с входом через Google. Инициализация Firebase вынесена в модуль `'auth.js'`, который проверяет наличие SDK, наличие конфигурации и повторно использует уже созданное приложение, если оно было инициализировано ранее. Это исключает конфликт повторной инициализации при переходах между страницами.

Страница профиля реализована в `'profile.js'`. После входа через Google пользователь получает доступ к своему имени, электронной почте и аватару. Дополнительные настройки хранятся в документе Firestore `'users/{uid}'`. К ним относятся отображаемое имя, URL Discord-вебхука, имя отправителя для Discord и цвет сообщений. Эти настройки используются другими частями сайта, прежде всего модулем бросков кубов.

Профиль выполняет роль связующего элемента между авторизацией и игровыми инструментами. Без входа пользователь не может полноценно использовать облачный редактор персонажей и персональные настройки бросков. После входа сайт получает идентификатор пользователя и может отделять его данные от данных других игроков на уровне структуры Firestore.

Важно, что в дипломной работе не приводятся реальные секретные значения вебхуков или токенов. Firebase-конфигурация описывается как техническая настройка подключения, а Discord-вебхук рассматривается как пользовательское значение, которое вводится на странице профиля и используется только для отправки результатов бросков.

3.3.2. Редактор персонажей и связь с базой знаний

Редактор персонажей реализован в `'character-editor.js'` и предназначен для создания, изменения и хранения игровых листов E'Magios Core. Он объединяет форму ввода, расчет производных характеристик, работу со школами и заклинаниями, сохранение в облаке, импорт и экспорт в JSON. В отличие от статических книг правил, редактор работает с пользовательскими данными, поэтому его функциональность связана с авторизацией.

В начале файла определяются основные переменные состояния: текущий пользователь, идентификатор редактируемого персонажа, список облачных персонажей, подписка на Firestore, флаги загрузки, признак несохраненных изменений и таймер автосохранения. Задержка автосохранения задана в 3000 миллисекунд. Это позволяет не отправлять данные в облако после каждого изменения поля, а группировать последовательность пользовательских действий.

Сохранение персонажа выполняется через коллекцию `'users/{uid}/characters'`. Перед сохранением редактор собирает данные формы, проверяет имя персонажа и наличие текущего пользователя, добавляет дату последнего изменения и записывает объект в Firestore. Если персонаж уже имеет идентификатор, документ обновляется; если идентификатора еще нет, создается новый документ.

Для отображения списка сохраненных персонажей используется подписка `'onSnapshot'`. Она отслеживает коллекцию персонажей текущего пользователя, сортирует документы по дате последнего изменения и перерисовывает список. Благодаря realtime-подписке пользователь видит актуальное состояние данных без ручной перезагрузки страницы.

Редактор также поддерживает экспорт и импорт персонажа в формате JSON. Экспорт создает файл из текущего состояния формы, а импорт читает выбранный пользователем JSON-файл и заполняет поля редактора. Эта возможность обеспечивает переносимость данных: игрок может сохранить локальную копию персонажа, передать ее ведущему или восстановить лист вне облачной синхронизации.

Редактор персонажей использует не только данные формы, но и справочники из базы знаний. При выборе школ и заклинаний загружаются данные из ``data/schools.json`` и ``data/spells.json``. Фильтры выбора устроены по тем же принципам, что и фильтры основной базы данных: пользователь может ограничить список по школе, типу, уровню, источнику, концентрации и другим параметрам.

Такой подход устраняет дублирование игровых данных. Заклинание не описывается отдельно в редакторе персонажей: редактор получает его из общего JSON-справочника, который также используется базой знаний. Если заклинание меняется в исходных материалах и затем обновляется через парсер, изменения становятся доступны и в базе, и в редакторе.

В описаниях заклинаний редактор выделяет выражения бросков, например ``2d4`` или ``3d8+2``, и превращает их в кликабельные элементы. Для этого используется регулярное выражение, которое находит допустимые формулы кубов и добавляет к ним атрибуты с исходным выражением и контекстом заклинания. При нажатии пользователь может выполнить бросок через общий модуль кубов.

Связь редактора с базой знаний делает лист персонажа не изолированной формой, а частью единой цифровой системы. Игрок выбирает сущности из актуальных справочников, получает доступ к описаниям и может сразу использовать игровые формулы в процессе партии.

3.3.3. Броски кубов и Discord-интеграция

Модуль бросков кубов реализован в ``common.js``, поскольку он используется на разных страницах сайта. Интерфейс включает панель истории, быстрые кнопки стандартных кубов, поле команды ``/roll``, настройки режима отображения и возможность выбрать персонажа. Если пользователь авторизован, модуль может загрузить список его персонажей и применять бонусы из листа к быстрым броскам.

Разбор выражений выполняется функцией `parseRollExpression`. Она принимает строку команды, удаляет префикс `/roll`, убирает пробелы, проверяет допустимые символы и разбивает выражение на сегменты по знакам `+` и `-`. Сегмент может быть броском кубов или числовым модификатором. Для кубов поддерживаются форматы `XdY`, `XdY*N` и `XdY/N`, а также ограниченный список граней: D2, D4, D6, D8, D10, D12, D20 и D100.

После разбора выражения выполняется генерация случайных значений для каждого куба, расчет промежуточных результатов и итоговой суммы. Результат добавляется в историю бросков. Если пользователь включил отправку в Discord и указал вебхук в профиле, результат дополнительно отправляется во внешний канал. Таким образом один модуль обслуживает как личные броски игрока, так и публичные броски для игровой группы.

Связь с персонажем реализована через выбор сохраненного листа или через текущий открытый лист редактора. В глобальную область экспортируется функция `getCurrentSheetCharacter`, возвращающая данные текущей формы. Благодаря этому быстрые броски могут учитывать актуальные бонусы персонажа даже во время редактирования листа.

Заключение

В ходе выполнения выпускной квалификационной работы был разработан веб-сайт E'Magios Core, представляющий собой многофункциональную цифровую платформу для сопровождения авторской настольной ролевой системы. Программный продукт объединяет справочный контент, структурированную базу данных игровых сущностей, редактор персонажей, систему бросков кубов, облачное хранение пользовательских данных и интеграцию с внешними сервисами. Реализованная архитектура, основанная на модульном разделении клиентской логики, использовании Firebase и автоматизированной обработке контента из Obsidian, демонстрирует гибкость и масштабируемость, что упрощает дальнейшее развитие проекта.

В процессе создания программного продукта были успешно выполнены этапы анализа, проектирования, разработки и тестирования, что позволило достичь соответствия поставленным целям и задачам. В ходе выполнения работы были достигнуты следующие результаты:

1. Проведено исследование предметной области цифровых инструментов для настольных ролевых систем;
2. Выполнен анализ аналогичных веб-проектов и справочных платформ;
3. Спроектирована архитектура веб-сайта, включающая навигацию, базу данных, пользовательские модули и механизм публикации контента;
4. Разработан веб-сайт с модулем базы данных, редактором персонажей, системой бросков кубов, авторизацией через Google и хранением данных в Firebase;
5. Реализована интеграция с Discord через вебхуки для отправки результатов бросков;
6. Организован механизм парсинга и преобразования контента из Obsidian в HTML- и JSON-форматы.

Список литературы

В процессе.